6 Trie, Radix Trie: Efficient Strings Search

This chapter covers

- Understanding why working with strings is different
- Introducing Trie[1] for efficient string search and indexing
- Introducing Radix Tree[2] as a memory-efficient evolution of Trie
- Using prefix trees to solve string-related problems
- Leveraging Tries to implement an efficient spell-checker

How many times have you sent a text, an email or twitted in a hurry, just to realize, a second later, that you had a typo? For me, it was too many times! Lately, however, we started to have a precious ally by default on our side in email clients, and browsers in general: spell-checkers! If you'd like to know more about how they work and how to implement them efficiently, this chapter is the right place to start.